

Experiment -1

Linux Programming Environment & Basic Commands

A) Setting up of Linux Programming Environment

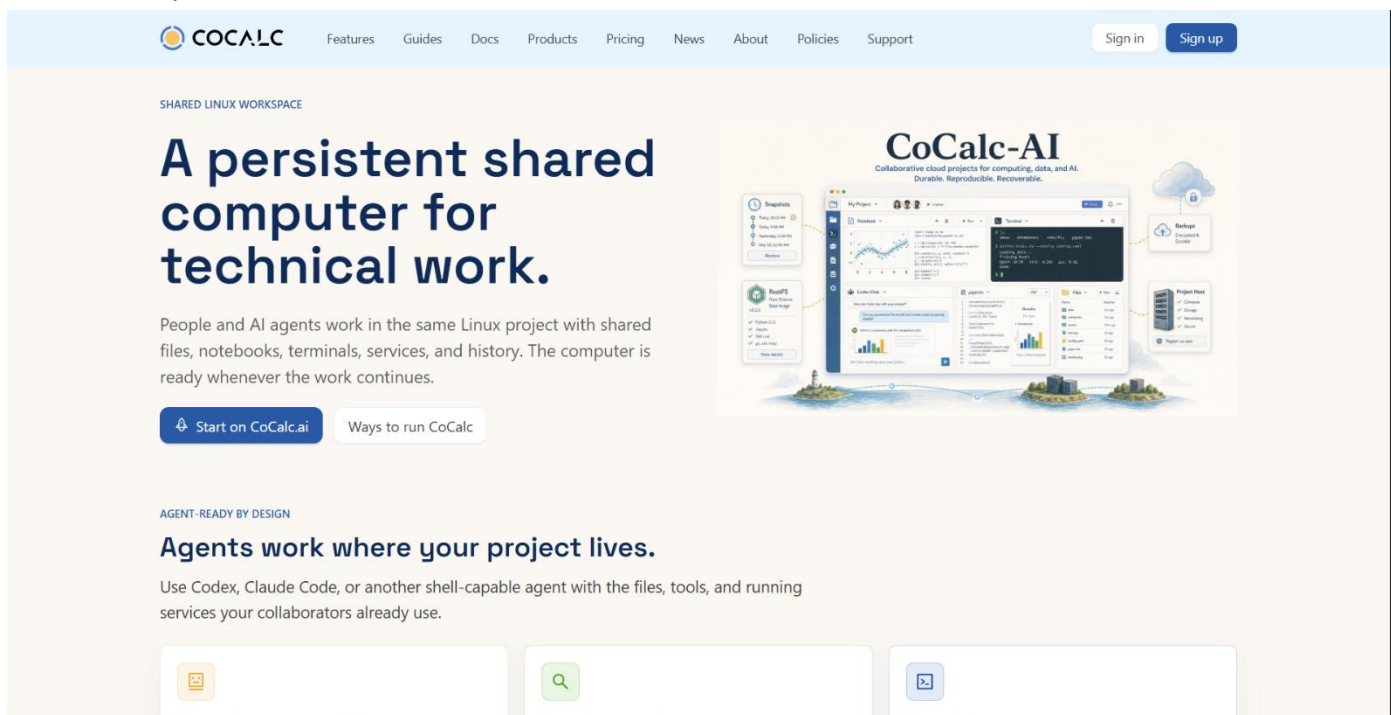
We can do C programming online or offline according to the needs. It is required for us to keep working on a Linux Environment.

Online: Cocalc.ai

CoCalc is an online cloud-based platform where you can use a Linux terminal and programming environment through a web browser without installing Linux on your computer.

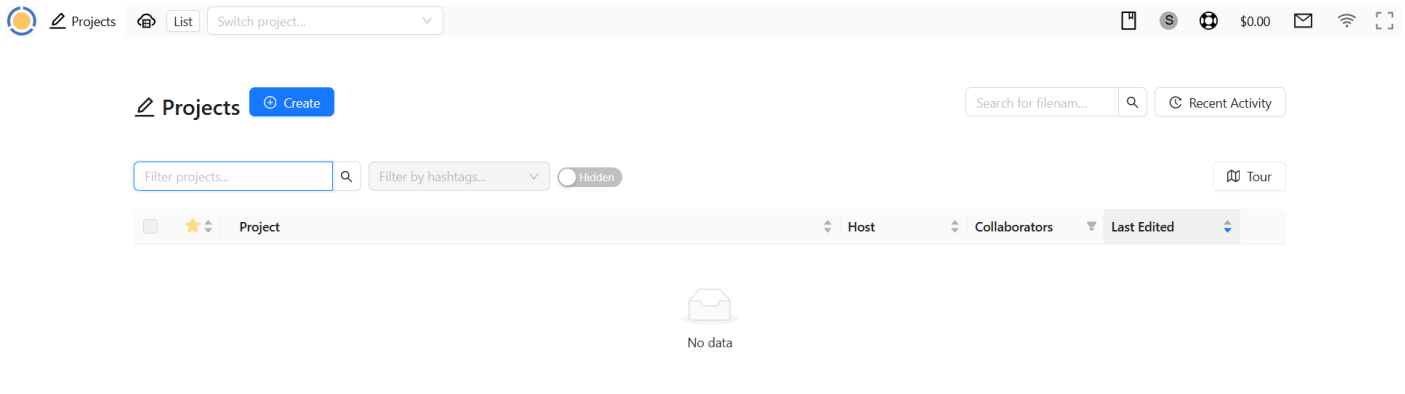
To use it we must follow the required steps-

STEP 1 - Open the CoCalc website in a web browser.

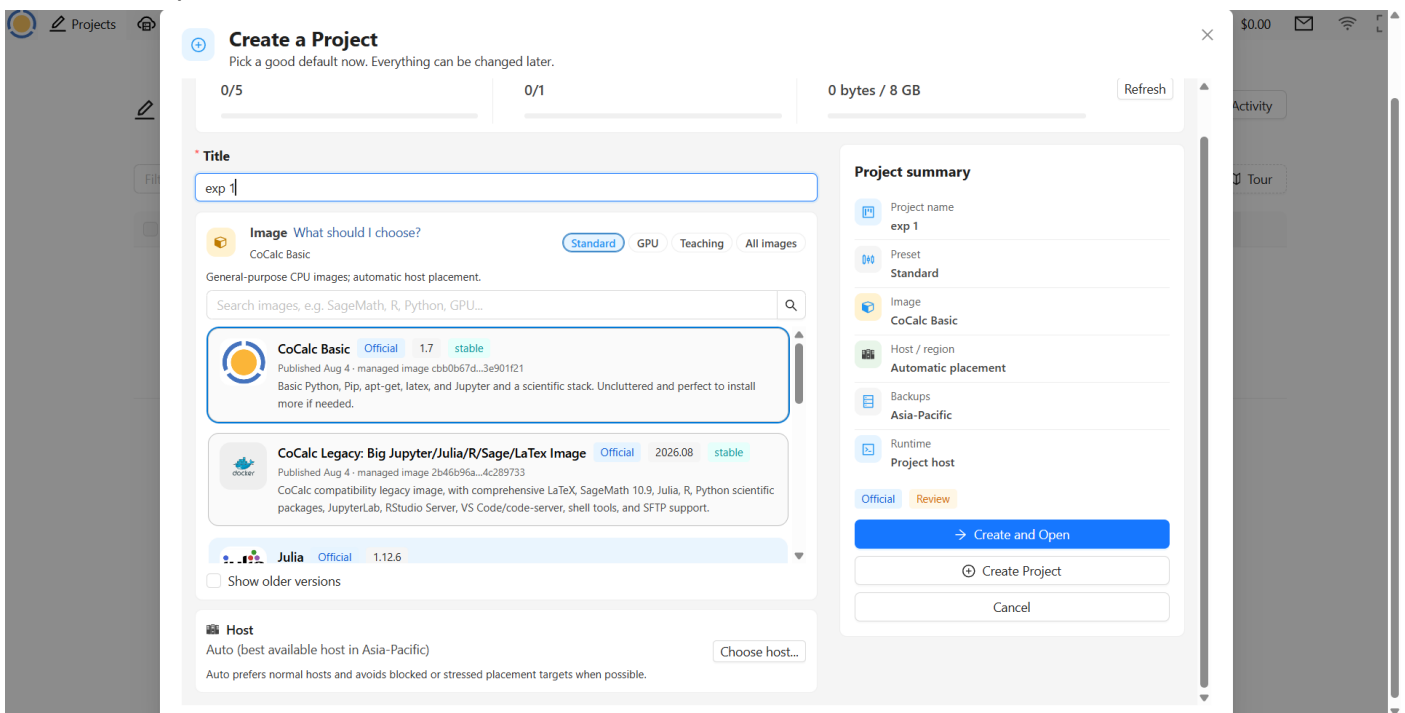


STEP 2 - Create an account or sign in.

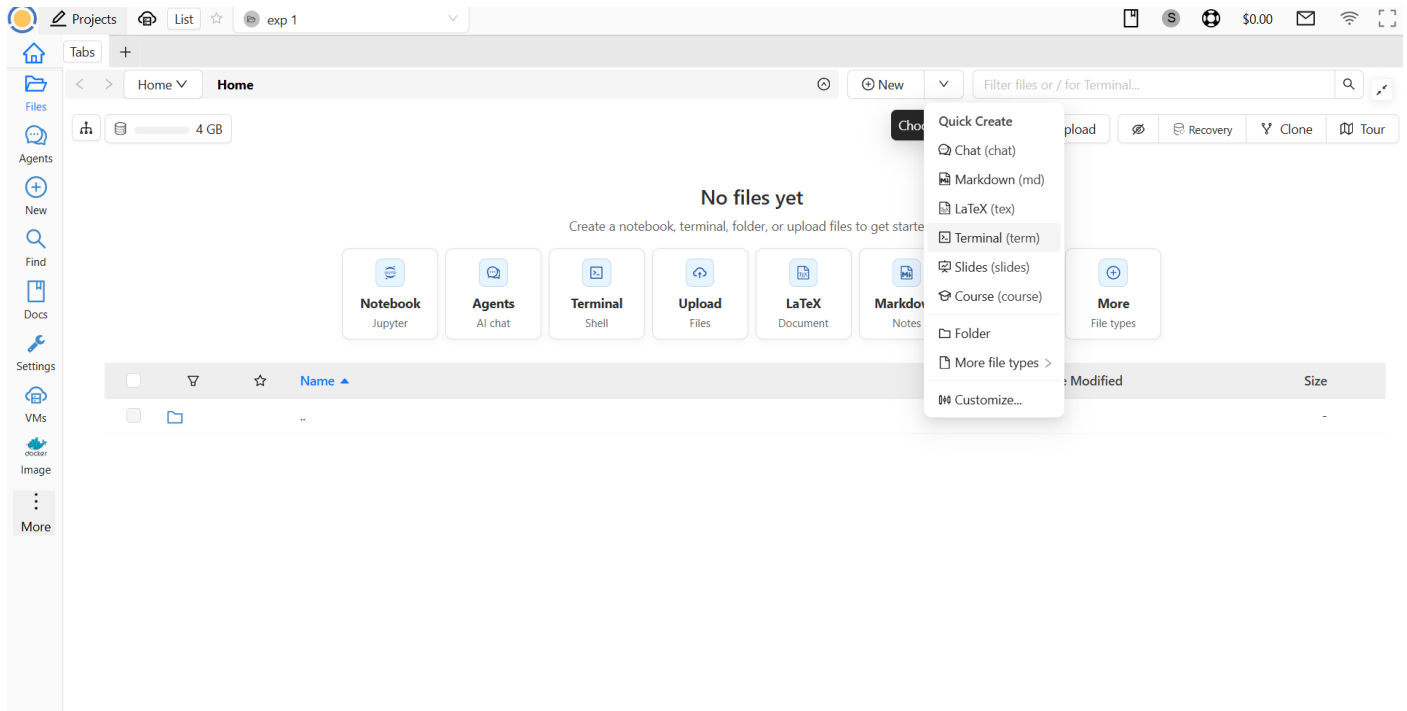
STEP 3 - Create a New Project by clicking on +create button in blue colour.



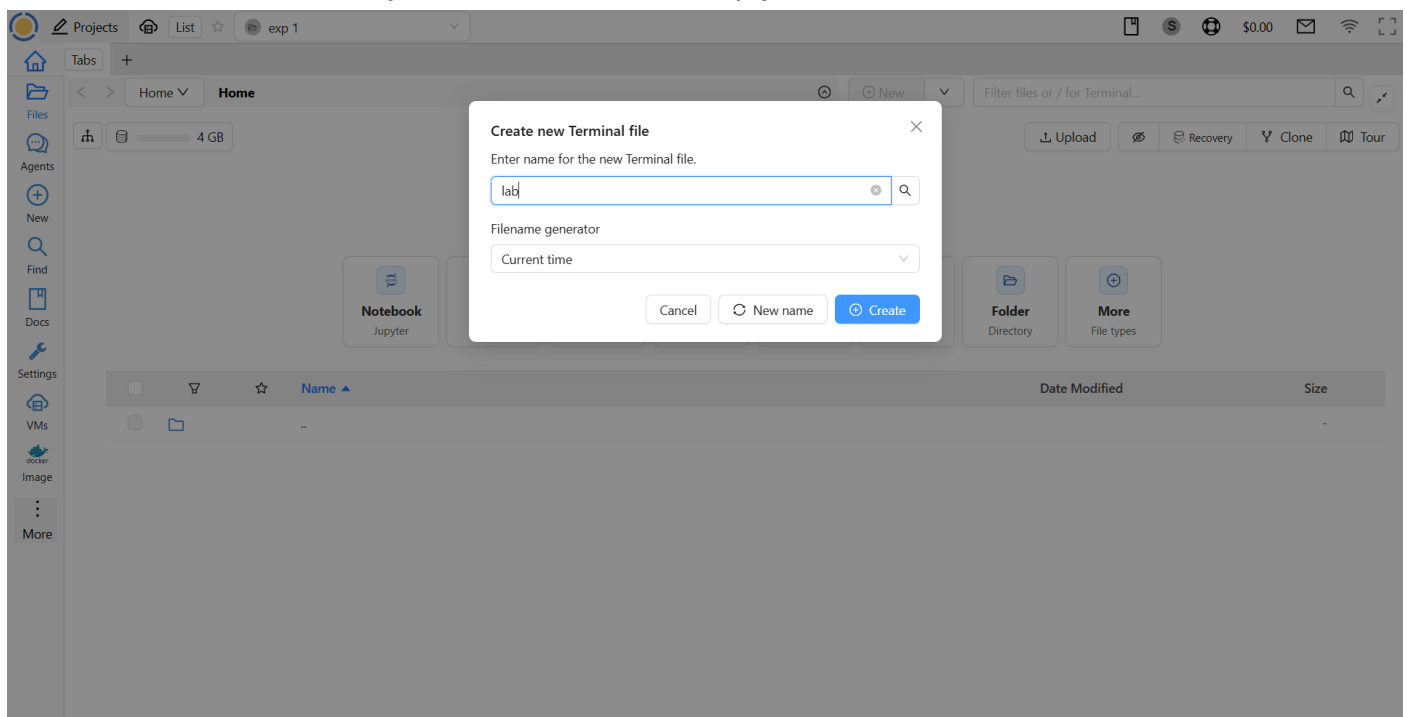
STEP 4 - Title the project and chose image you wish to operate provided there then simply click on create and open.



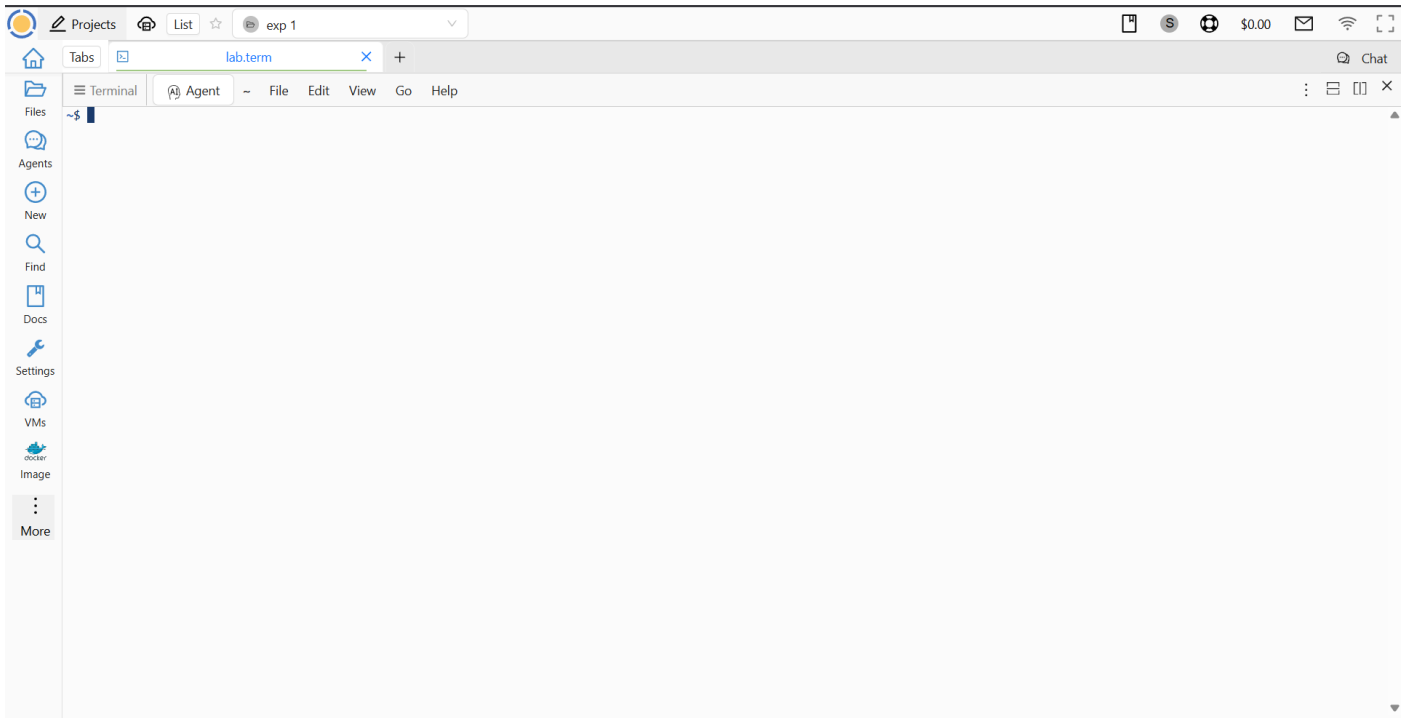
STEP 5 - Open Terminal(Linux Terminal) by clicking quick create option inside the project.



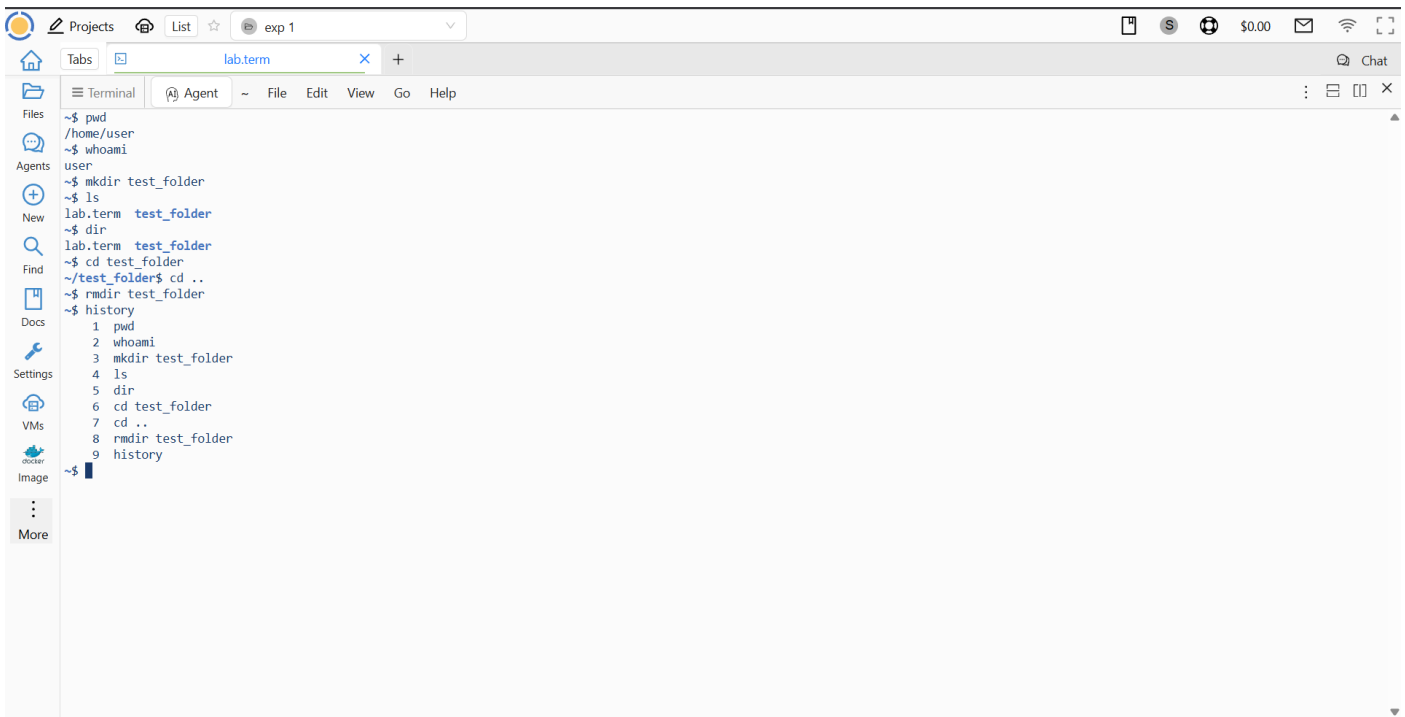
STEP 6 - Enter the name for your terminal file and simply create it.



STEP 7 - Now the Terminal that we just created provides a Linux environment.



STEP 8 - Now in this terminal file use commands such as **pwd**, **whoami**, **mkdir**, **ls**, **dir**, **cd**, **cd ..**, **rmdir**, **history**, **clear**.

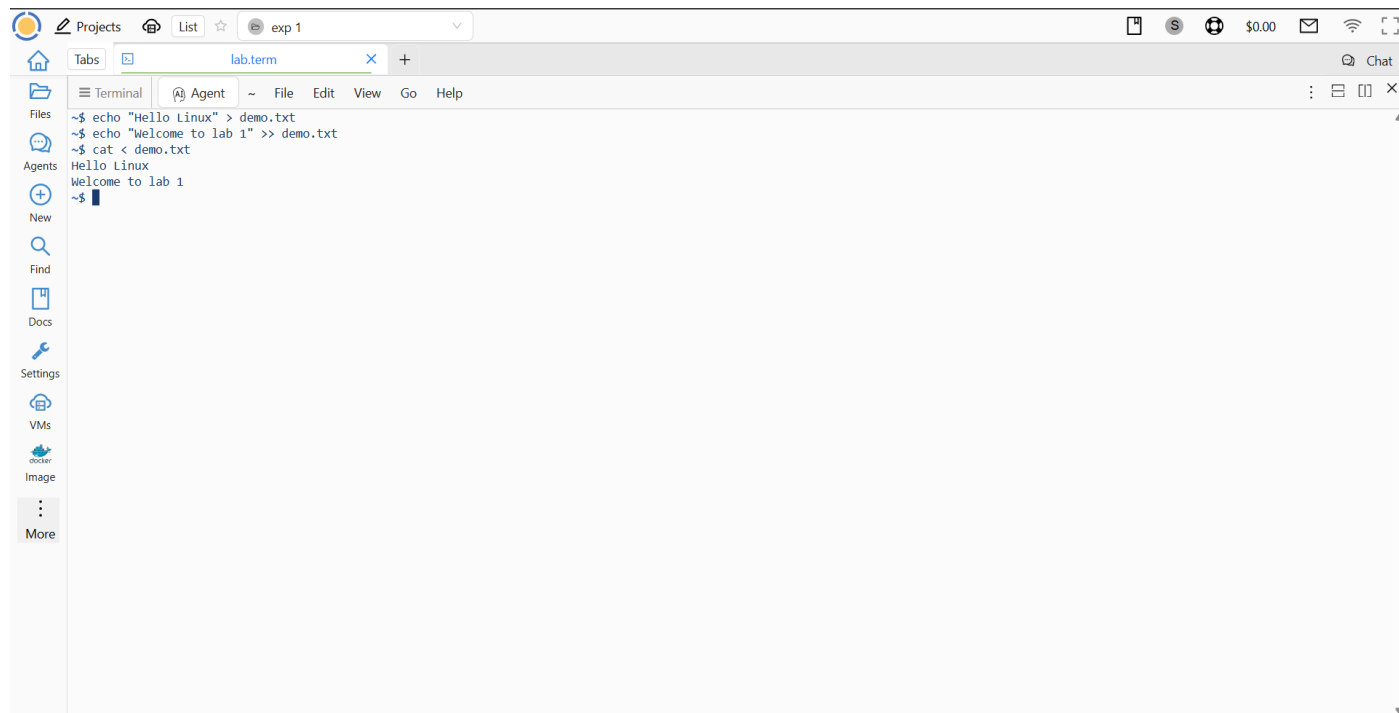


Above commands are called **TERMINAL BASICS** and each command has its own function such as-

1. **pwd** – It shows current working directory.
2. **whoami** – It shows current logged-in user.
3. **mkdir** - This command creates a new directory(In above picture the test_folder is new directory).

4. **ls** – The command will list current folders and files in it.
5. **cd** – This command helps in entering a new directory that we created(In above picture the command `cd test_folder` shows it).
6. **cd ..** – This command helps in entering parent directory.
7. **rmdir** – This command will delete empty folder(here `test_folder`).
8. **history** – This command will show history of the all commands that have been run.
9. **man** – This command will open manual/help page to close it press `q` from keyboard.
10. **clear** – This command will clear Terminal screen.

STEP 9 – Now we will use **Redirection commands** such as : `>`, `>>`, `<`. These are used to direct the standard input and output streams to or from files instead of the terminal screen or keyboard.



The screenshot shows a terminal window with the following commands and output:

```
~$ echo "Hello Linux" > demo.txt
~$ echo "Welcome to lab 1" >> demo.txt
~$ cat < demo.txt
Hello Linux
Welcome to lab 1
~$
```

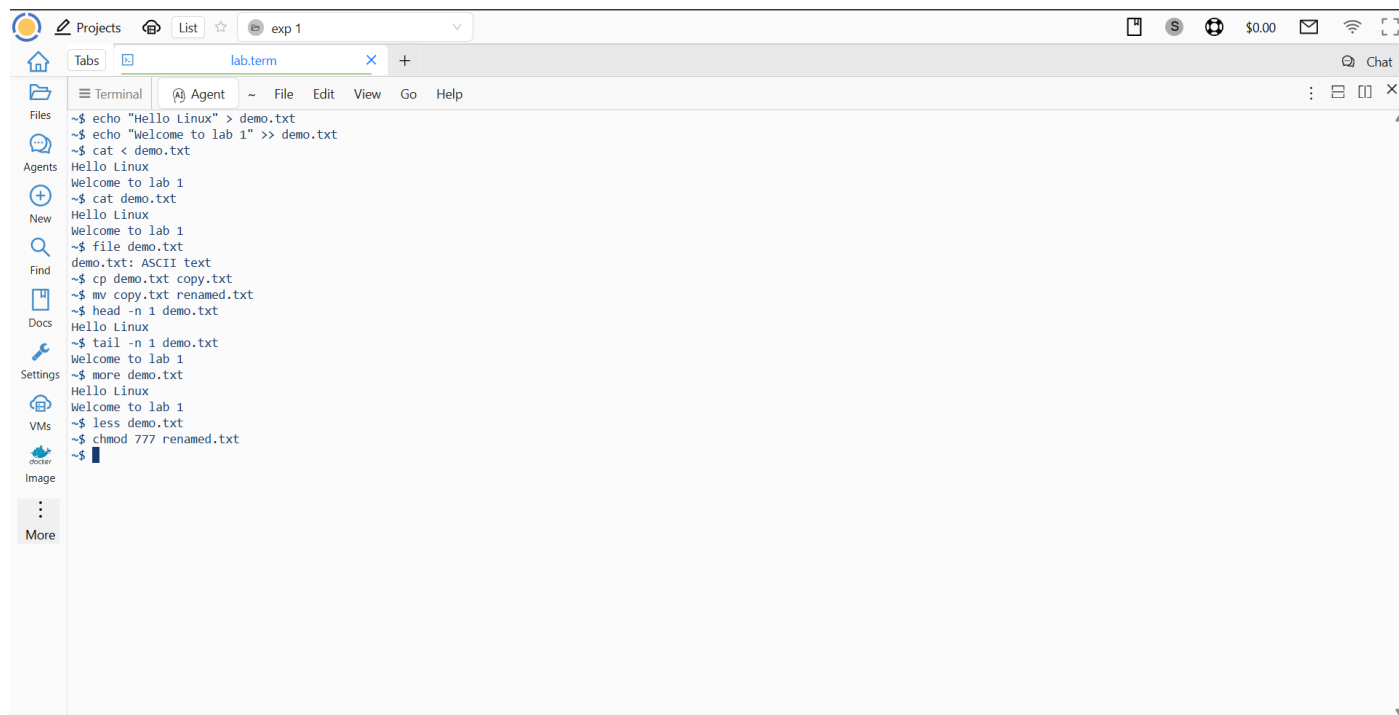
The terminal window has a sidebar on the left with icons for Files, Agents, New, Find, Docs, Settings, VMs, Docker, Image, and More. The top bar shows 'lab.term' and various system icons.

Above commands are called **Redirections** each and every command has its own function such as:

1. **Output Redirection (>)**: Directs the command output to file, creating a new file or overwrite existing content (In above picture - **echo "Hello Linux" > demo.txt**).
2. **Append Redirection (>>)**: Adds the command output to the end of an existing file without deleting its current content (In above picture – **echo "Welcome to lab 1" >> demo.txt**).
3. **Input Redirection (<)**: Passes the contents of file as input to a command instead of taking input from keyboard (In above picture – **cat < demo.txt**).

And as a result the command have given the result as shown in above picture.

STEP 10 – As we have seen that in step 9 we have used some of file operation command and now in this step we will use more **File Operation Commands** such as: **vi, file, rm, cp, mv, cat, more, less, head, tail, chmod**. File operation commands are used to create, view, modify, copy, move, set permissions, and delete files in the Linux file system.



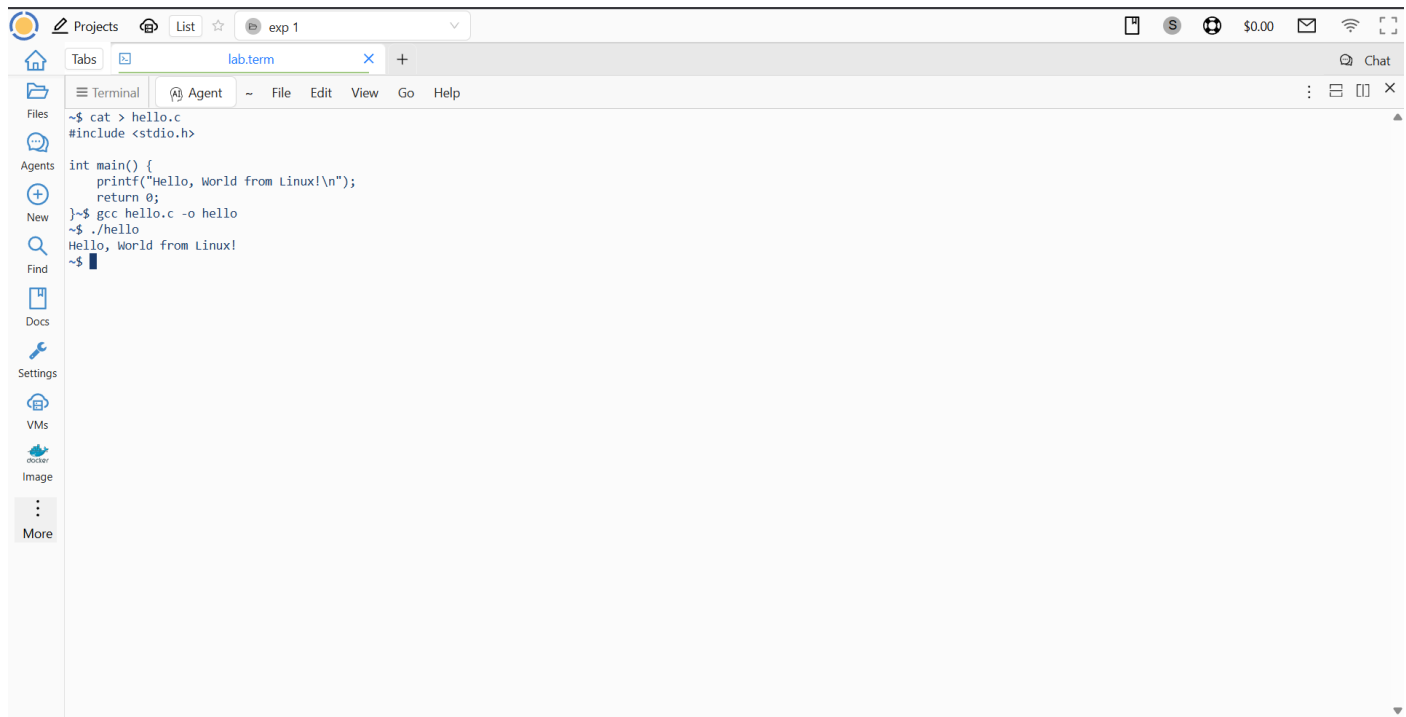
The screenshot shows a terminal window with a sidebar on the left containing icons for Files, Agents, New, Find, Docs, Settings, VMs, Docker, and Image. The terminal output is as follows:

```
~$ echo "Hello Linux" > demo.txt
~$ echo "Welcome to lab 1" >> demo.txt
~$ cat < demo.txt
Hello Linux
Welcome to lab 1
~$ cat demo.txt
Hello Linux
Welcome to lab 1
~$ file demo.txt
demo.txt: ASCII text
~$ cp demo.txt copy.txt
~$ mv copy.txt renamed.txt
~$ head -n 1 demo.txt
Hello Linux
~$ tail -n 1 demo.txt
Welcome to lab 1
~$ more demo.txt
Hello Linux
Welcome to lab 1
~$ less demo.txt
~$ chmod 777 renamed.txt
~$
```

Above commands are called **File command** and each command has its own function such as:

1. **vi**: A command-line text editor used to create and edit text files.
2. **file**: Determines and displays the file type (e.g., ASCII text, directory, executable). In above picture **file demo.txt**
3. **cat**: Concatenates and displays the entire content of file on the terminal screen. (In above picture **cat demo.txt**)
4. **more**: Views file content page-by-page, allowing forward navigation using the Spacebar. (In above picture **more demo.txt**)
5. **less**: An advanced viewer that allows both forward and backward scrolling through large files (q to exit). (In above picture **less demo.txt**)
6. **head**: Displays the first few lines of a file (default is 10 lines). (In above picture **head -n 1 demo.txt** will give only one and first line as specified by command given)
7. **tail**: Displays the last few lines of a file (default is 10 lines). (In above picture **tail -n 1 demo.txt** will give only one and last line as specified by the command given)
8. **cp**: Copies files or directories from one location to another. (In above picture **cp demo.txt copy.txt**)
9. **mv**: Moves or renames files and directories. (In above picture **mv copy.txt renamed.txt**)
10. **chmod**: Changes the access permissions (read, write, execute) of a file or folder. (In above picture **chmod 777 renamed.txt**)
11. **rm**: Removes (deletes) files from the file system.(only give command as **rm demo.txt**)

STEP 11 – Now we know major commands so we can run a simple ‘c’ program in the terminal.



```
~$ cat > hello.c
#include <stdio.h>

int main() {
    printf("Hello, World from Linux!\n");
    return 0;
}~$ gcc hello.c -o hello
~$ ./hello
Hello, World from Linux!
~$
```

In above picture we completed a c program successfully the steps are as follows:

1. `cat > hello.c`
2. `#include <stdio.h>`
`int main() {`
`printf (Hello, World from Linux!\n");`
`return 0;`
`}`
3. Save the command by **Ctrl+D**.
4. Now for compilation type:
`gcc hello.c -o hello`
5. Now simply run the command by
`./hello`
6. The output will be – **Hello, World from Linux! (As in above picture)**

Offline:

Methods: Dual boot, Single boot, Bootable Pendrive, VM

a) Dual Boot

In a dual-boot system, Windows and Linux are installed on the same computer.

When the computer starts, you get a menu asking which operating system you want to use.

For example:

Windows 11

Kali Linux

You select whichever OS you want.

Advantage: Linux gets direct access to the computer's hardware and generally performs very well.

Disadvantage: Partitioning the disk incorrectly can cause data loss, so beginners need to be careful.

b) Single Boot

In a single-boot system, only one operating system is installed on the computer.

For example, if you install Kali and remove Windows, the computer will directly start Kali.

Advantage: All system resources are available to Linux.

Disadvantage: You cannot directly use Windows unless you reinstall it or use virtualization.

c) Bootable Pendrive / Live USB

A Linux ISO file is written to a USB pendrive, making it bootable.

You can then start your computer from the USB and run Linux.

There are two possibilities:

* Live mode: Linux runs from the USB without permanently installing it.

* Installation: Linux can be installed onto the computer from the USB.

Advantage: Useful for testing Linux or installing it.

Disadvantage: Running Linux directly from a USB is generally slower than running it from an internal SSD.

d) Virtual Machine (VM)

A Virtual Machine allows us to run another operating system inside our existing operating system.

For example:

Windows → Oracle VirtualBox → Kali Linux

Our Windows system continues running normally while Ubuntu runs inside a virtual machine.

This is the method recommended for our experiment because we don't need to modify our Windows installation or partition our SSD.

1. Virtual Machine Manager (Hypervisor)

A **hypervisor** is software or firmware that creates and manages virtual machines.

It allows one physical computer to run multiple operating systems virtually.

There are two major types.

Type-1 Hypervisor — Bare Metal

A Type-1 hypervisor runs **directly on the physical hardware**, without requiring a conventional host operating system underneath it.

Examples:

* VMware ESXi

* Microsoft Hyper-V Server

* Xen

Simple diagram:

Hardware

↓

Type-1 Hypervisor

↓

Virtual Machines

Where is it used?

Type-1 hypervisors are commonly used in **servers, data centres and enterprise environments** because of their performance and efficient resource management.

Type-2 Hypervisor — Hosted

A Type-2 hypervisor runs on top of an existing operating system.

In our case:

Hardware

↓

Windows

↓

Oracle VirtualBox

↓

Kali Linux VM

Examples:

* Oracle VirtualBox

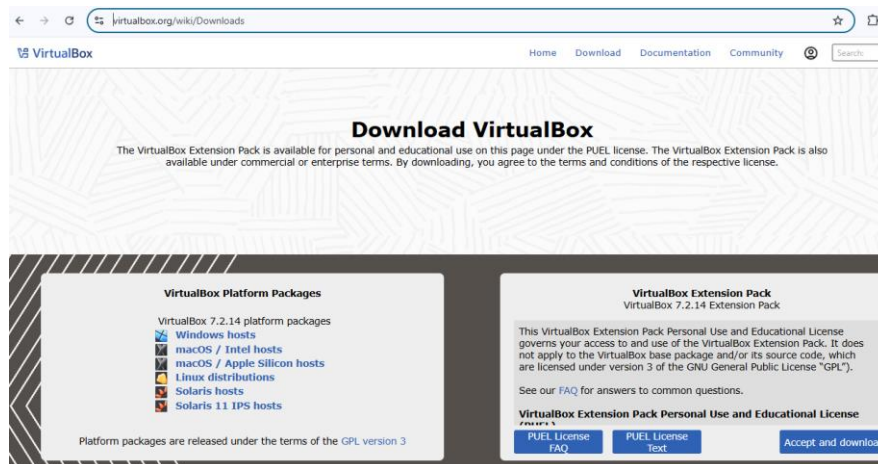
* VMware Workstation

* VMware Fusion

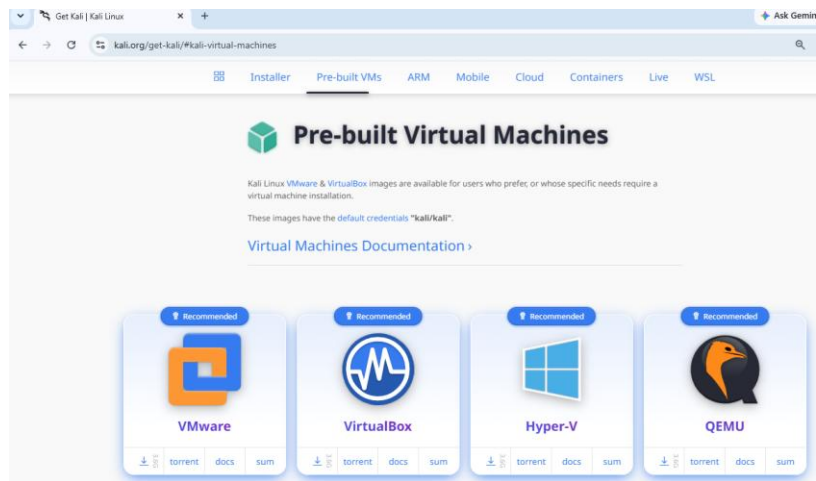
We will use Oracle VirtualBox.

2. Installation of VMM & VM

Step-1: Download Oracle Virtual Box for “Windows hosts”

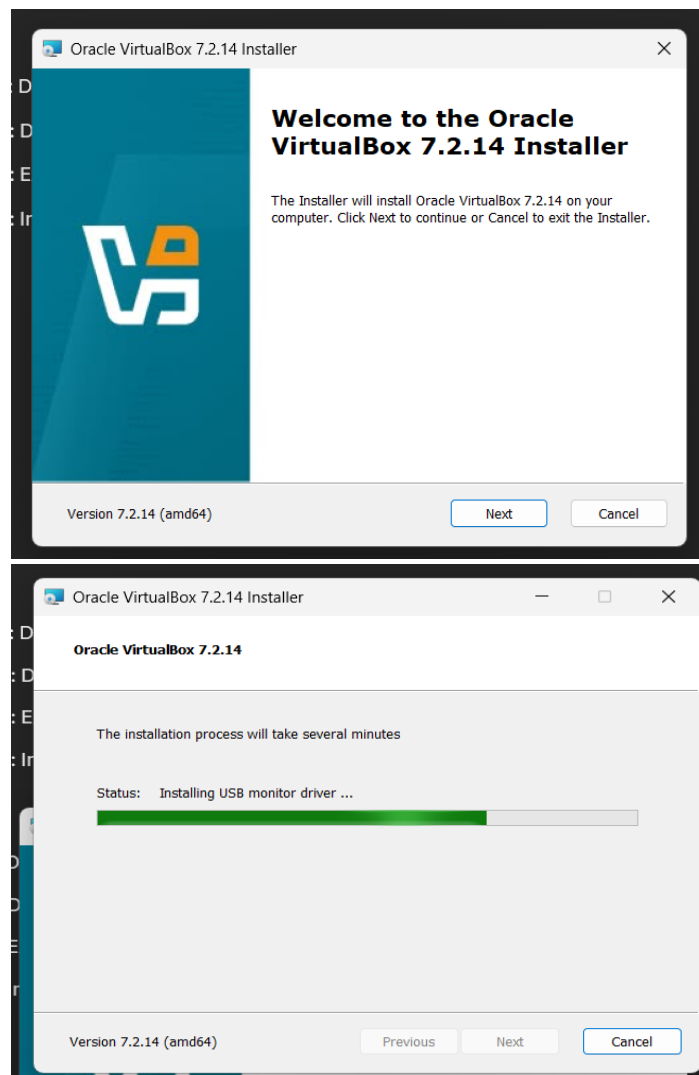


Step-2: Download Pre-built Kali Linux VM for Virtualbox

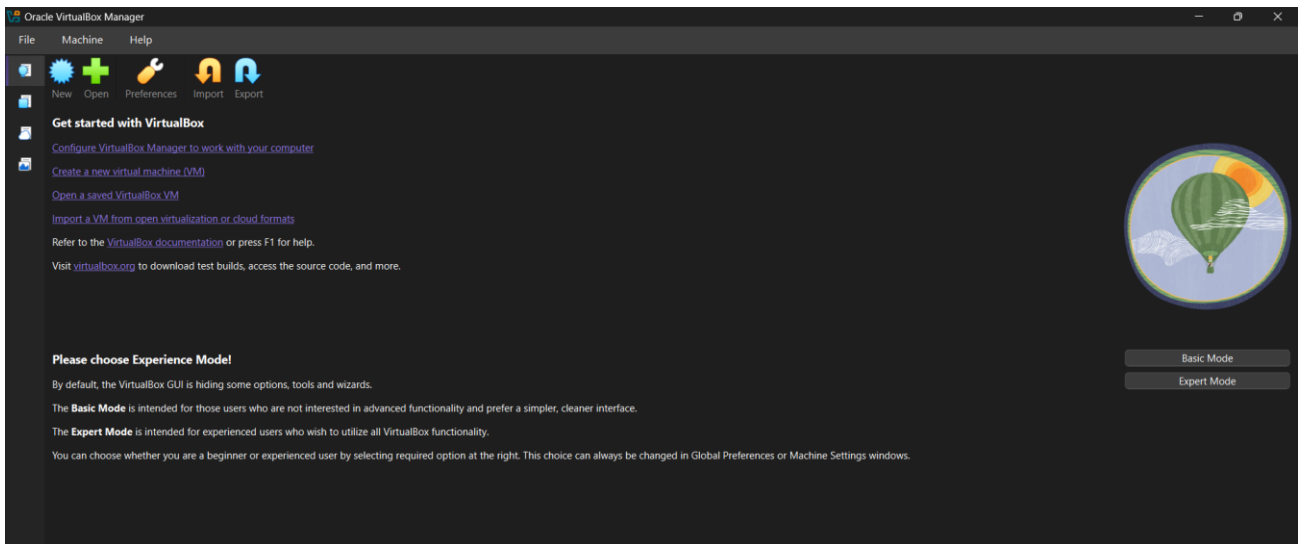


Step-3: Extracted the Kali Linux downloaded file

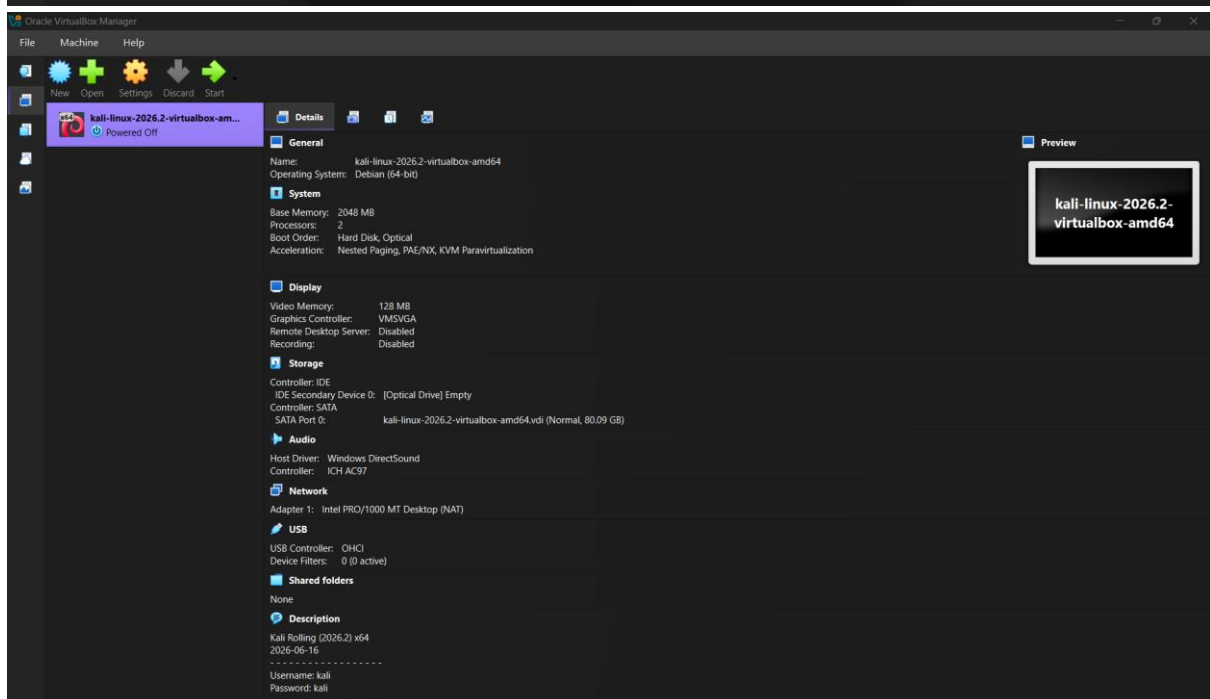
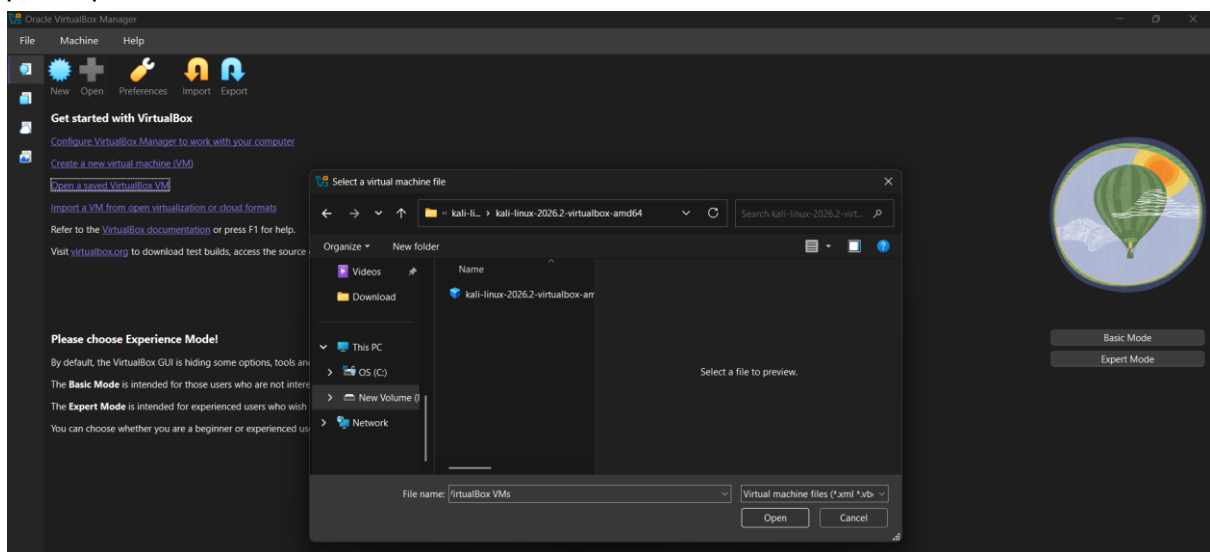
Step-4: Install Virtualbox



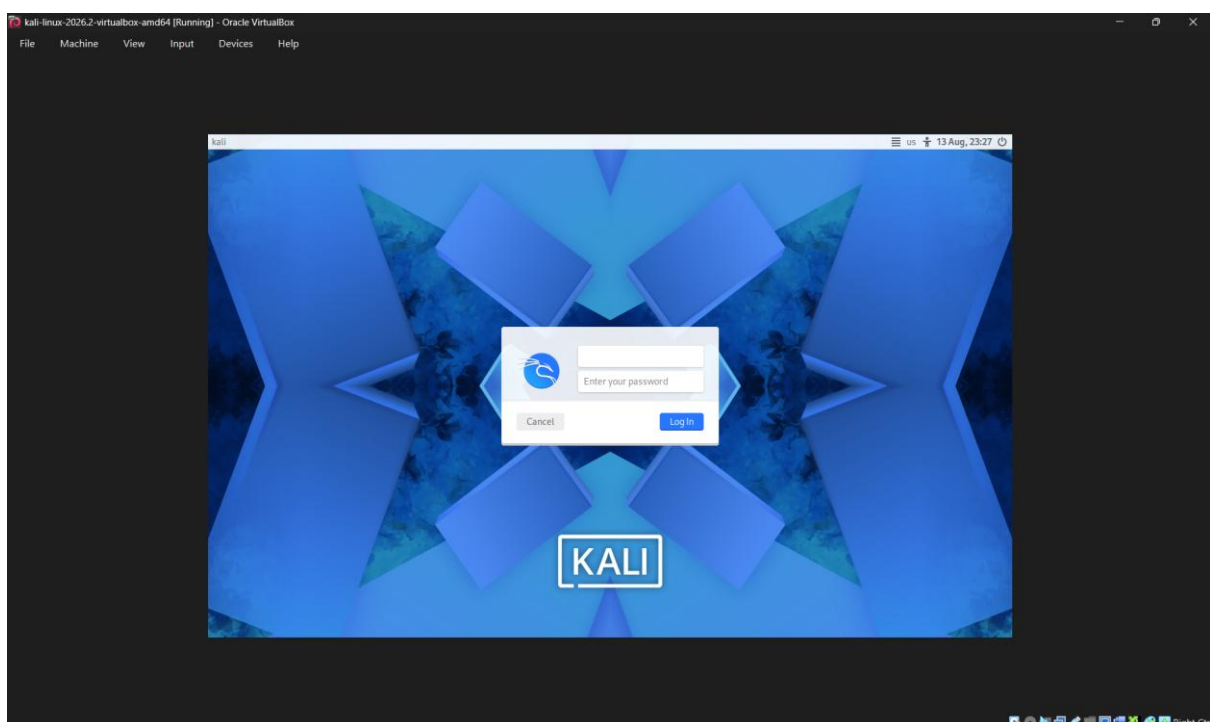
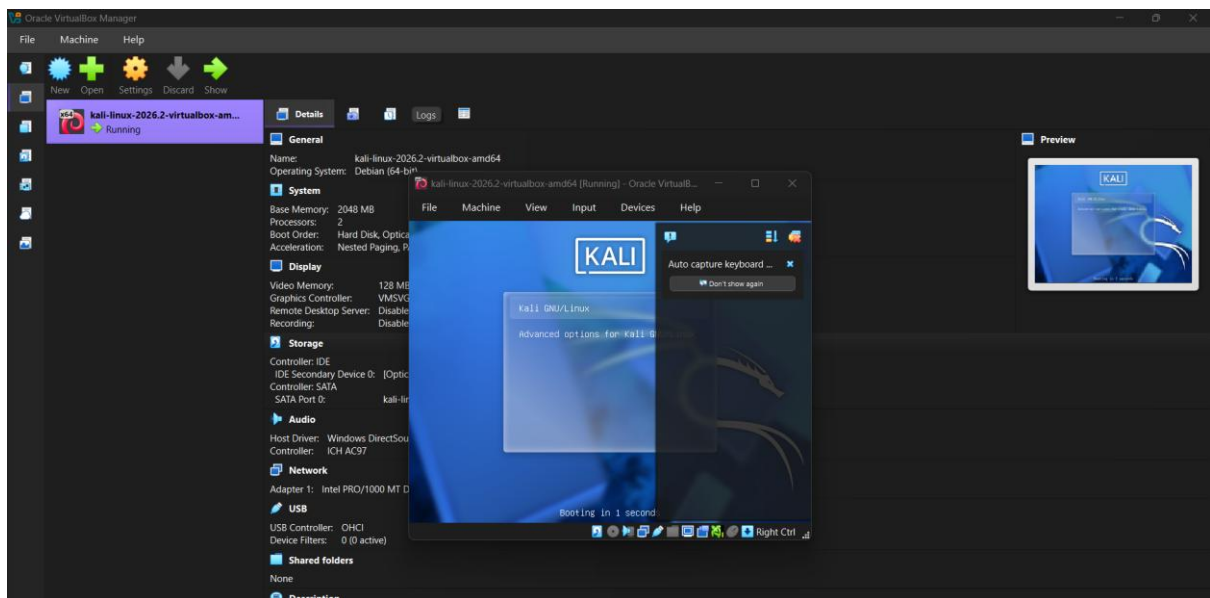
Step-5: Open Virtual box



Step-6: Open the “saved VM”



Step-7: Start the VM



Thus, the installation of VMM (Hypervisor Type-2 Oracle VirtualBox) & VM (Kali Linux) is completed.

Step-8: Open terminal and create the following simple program, compile and execute to understand the working of the environment.

LINUX COMMANDS

1.TERMINAL BASICS COMMANDS

1. **pwd** – It shows current working directory.
2. **whoami** – It shows current logged-in user.
3. **mkdir** - This command creates a new directory.
4. **ls** – The command will list current folders and files in it.
5. **cd** – This command helps in entering a new directory that we created.
6. **cd ..** – This command helps in entering parent directory.
7. **rmdir** – This command will delete empty folder.

8. **history** – This command will show history of the all commands that have been run.
9. **man** – This command will open manual/help page to close it press q from keyboard.
10. **clear** – This command will clear Terminal screen.

2.FILES OPERATION COMMANDS

1. **vi**: A command-line text editor used to create and edit text files.
2. **file**: Determines and displays the file type (e.g., ASCII text, directory, executable).
3. **cat**: Concatenates and displays the entire content of file on the terminal screen.
4. **more**: Views file content page-by-page, allowing forward navigation using the Spacebar
5. **less**: An advanced viewer that allows both forward and backward scrolling through large files (q to exit).
6. **head**: Displays the first few lines of a file (default is 10 lines).
7. **tail**: Displays the last few lines of a file (default is 10 lines).
8. **cp**: Copies files or directories from one location to another.
9. **mv**: Moves or renames files and directories.
10. **chmod**: Changes the access permissions (read, write, execute) of a file or folder
11. **rm**: Removes (deletes) files from the file system.

3.REDIRECT COMMANDS

1. **Output Redirection (>)**: Directs the command output to file, creating a new file or overwrite existing content .
2. **Append Redirection (>>)**: Adds the command output to the end of an existing file without deleting its current content .
3. **Input Redirection (<)**: Passes the contents of file as input to a command instead of taking input from keyboard.

```

Session Actions Edit View Help
(ranveer@ok)-[~]
$ pwd
/home/ranveer

(ranveer@ok)-[~]
$ whoami
ranveer

(ranveer@ok)-[~]
$ mkdir exam

(ranveer@ok)-[~]
$ ls
AcnZIUnm.html  Desktop  Downloads  hello.c  Kismet-20260202-07-20-25-1.kismet  pentbox  PPPwn  SpbmQCgo.jpeg  Videos
BlueSpy       dndhpl8c.jpeg  exam      hhubWPLW.jpeg  Music    prAXRGdy.html  Templates  zphisher
CamPhish     Documents     FTifToCh.html  IFFDvKdW.jpeg  ormuFDqX.jpeg  trojan.exe

(ranveer@ok)-[~]
$ cd exam

(ranveer@ok)-[~/exam]
$ cd ..

(ranveer@ok)-[~]
$ rmdir test
rmdir: failed to remove 'test': No such file or directory

(ranveer@ok)-[~]
$ rmdir exam

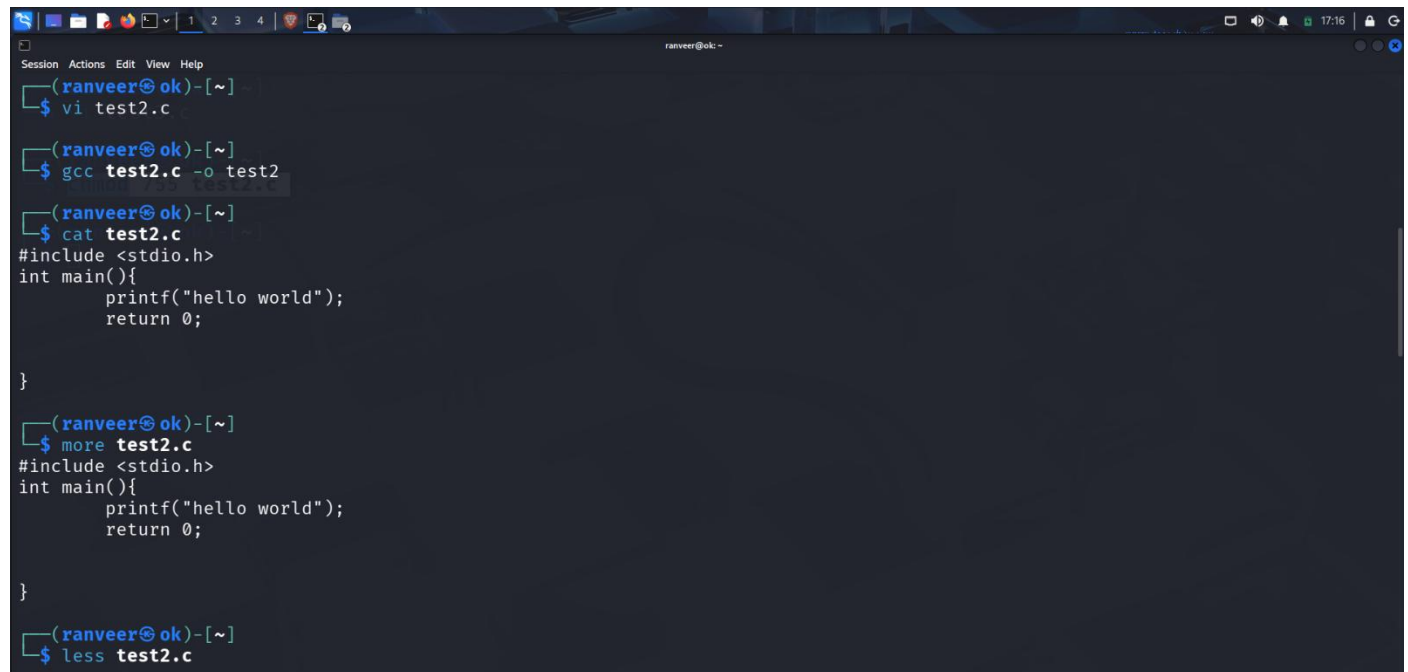
(ranveer@ok)-[~]
$ man
What manual page do you want?
For example, try 'man man'.

```

History command :

```
361 clear
362 cd ~/
363 clear
364 pwd
365 whoami
366 mkdir exam
367 ls
368 cd exam
369 cd ..
370 rmdir test
371 rmdir exam
372 man
```

```
(ranveer@ok)-[~]
$
```



A screenshot of a terminal window with a dark background and a keyboard graphic. The terminal shows a series of commands and their outputs. The user starts by opening a file named 'test2.c' in the 'vi' editor. They then compile the file using 'gcc test2.c -o test2'. Next, they view the contents of 'test2.c' using 'cat', which shows a simple C program that prints 'hello world'. Finally, they use 'more' and 'less' to view the file again. The terminal prompt is '(ranveer@ok)-[~]'.

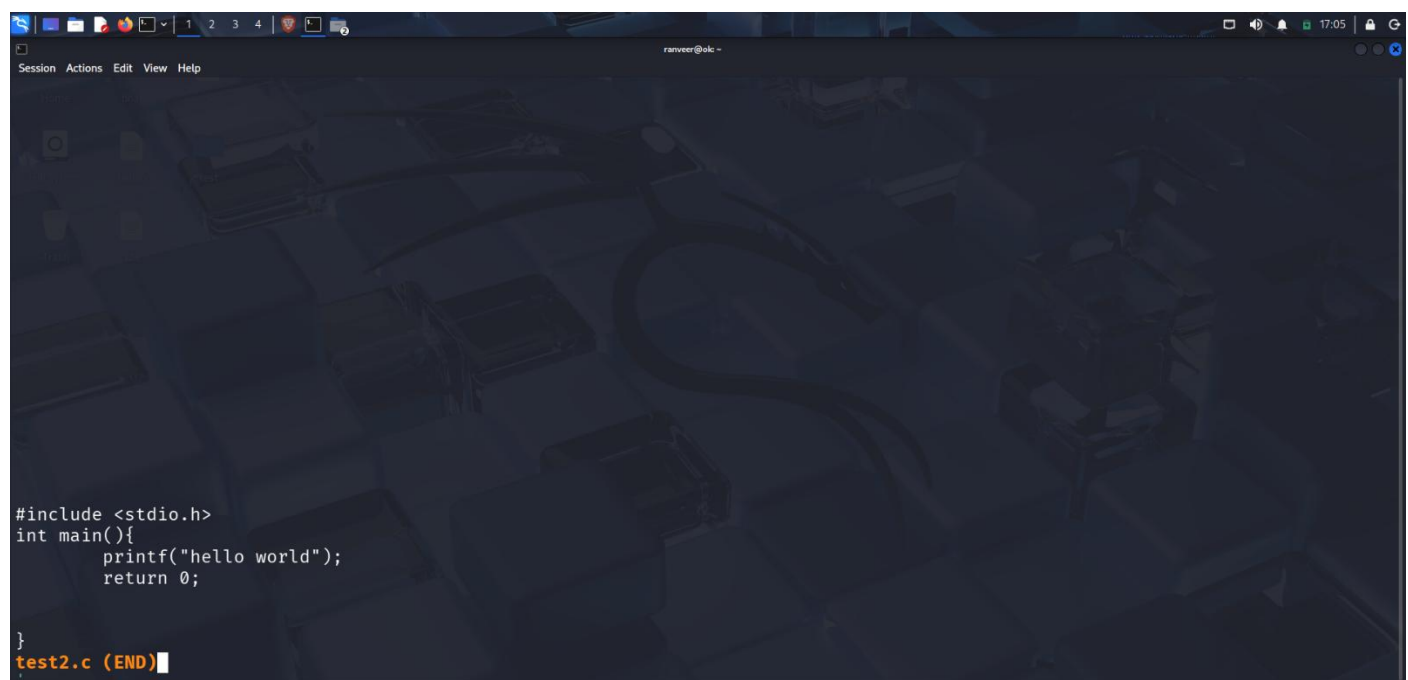
```
(ranveer@ok)-[~]
$ vi test2.c

(ranveer@ok)-[~]
$ gcc test2.c -o test2

(ranveer@ok)-[~]
$ cat test2.c
#include <stdio.h>
int main(){
    printf("hello world");
    return 0;
}

(ranveer@ok)-[~]
$ more test2.c
#include <stdio.h>
int main(){
    printf("hello world");
    return 0;
}

(ranveer@ok)-[~]
$ less test2.c
```



A screenshot of a terminal window showing the end of a file. The terminal displays the contents of 'test2.c' again, which is a simple C program that prints 'hello world'. The file ends with 'test2.c (END)'. The terminal prompt is '(ranveer@ok)-[~]'.

```
#include <stdio.h>
int main(){
    printf("hello world");
    return 0;
}
test2.c (END)
```

```
Session Actions Edit View Help
ranveer@ok: ~
$ less test2.c
(ranveer@ok)-[~]
$ head test2.c
#include <stdio.h>
int main(){
    printf("hello world");
    return 0;
}

(ranveer@ok)-[~]
$ tail test2.c
#include <stdio.h>
int main(){
    printf("hello world");
    return 0;
}

(ranveer@ok)-[~]
$ cp test2.c test3.c

(ranveer@ok)-[~]
$ ls
AcnZIUnm.html  Documents  IFFDvKdW.jpeg  pentbox-1.8  SpbmQCgo.jpeg  test.c
```

```
Session Actions Edit View Help
ranveer@ok: ~
$ ls
AcnZIUnm.html  Documents  IFFDvKdW.jpeg  pentbox-1.8  SpbmQCgo.jpeg  test.c
BlueSpy        Downloads  Kismet-20260202-07-20-25-1.kismet  Pictures      Templates      trojan.exe
CamPhish       FTifToCh.html  Music          PPPwn        test2          Videos
Desktop        hello.c        ormuFDqX.jpeg  prAXRGdy.html  test2.c       zphisher
dNdhlBc.jpeg   hhubWPLW.jpeg  pentbox        Public        test3.c

(ranveer@ok)-[~]
$ mv test3.c test4.c

(ranveer@ok)-[~]
$ chmod 755 test1.c
chmod: cannot access 'test1.c': No such file or directory

(ranveer@ok)-[~]
$ chmod 755 test2.c

(ranveer@ok)-[~]
$ ls -l test2.c
-rwxr-xr-x 1 ranveer ranveer 70 Aug 18 17:03 test2.c

(ranveer@ok)-[~]
$ rm test3.c
rm: cannot remove 'test3.c': No such file or directory
```



```
ranveer@ok: ~  
$ chmod 755 test2.c  
$ ls -l test2.c  
-rwxr-xr-x 1 ranveer ranveer 70 Aug 18 17:03 test2.c  
$ rm test3.c  
rm: cannot remove 'test3.c': No such file or directory  
$ rm test4.c\  
>  
$ rm test4.c  
rm: cannot remove 'test4.c': No such file or directory  
$ ls  
AcnZIUnm.html  dNdhplBc.jpeg  hello.c  Music  Pictures  SpbmQCgo.jpeg  test.c  
BlueSpy        Documents      hhubWPLW.jpeg  ormuFDqX.jpeg  PPPwn  Templates  trojan.exe  
CamPhish       Downloads     IFFDvKdW.jpeg  pentbox        prAXRGdy.html  test2      Videos  
Desktop        FTifToCh.html Kismet-20260202-07-20-25-1.kismet  pentbox-1.8   Public  test2.c     zphisher
```

```
ranveer@ok: ~  
$ ls  
AcnZIUnm.html  Documents  IFFDvKdW.jpeg  pentbox-1.8  SpbmQCgo.jpeg  test.c  
BlueSpy        Downloads  Kismet-20260202-07-20-25-1.kismet  Pictures      Templates  trojan.exe  
CamPhish       FTifToCh.html  Music  ormuFDqX.jpeg  PPPwn  test2      Videos  
Desktop        hello.c      pentbox        prAXRGdy.html  test2.c  test3.c  zphisher  
dNdhplBc.jpeg  hhubWPLW.jpeg  
$ mv test3.c test4.c  
$ chmod 755 test1.c  
chmod: cannot access 'test1.c': No such file or directory  
$ chmod 755 test2.c  
$ ls -l test2.c  
-rwxr-xr-x 1 ranveer ranveer 70 Aug 18 17:03 test2.c  
$ rm test3.c  
rm: cannot remove 'test3.c': No such file or directory
```

Submitted By: Ranveer Jat
SAP ID: 590032569
Batch 16, semester 1
B.tech - CSE
School of computer science